

SE 413/513: Software Testing

Assignment 1: Testing a Practical Course Registration System

Total Points: 10

Submission: GitHub repository and branch

Platform: Python with `pytest` is the default and assumed throughout this document. A different programming language/testing framework may be used only with prior instructor approval, and must still demonstrate the equivalent concepts (fixtures/setup-teardown, parametrized tests, positive/negative tests, markers/tags).

Coverage: Full statement/branch coverage is **not** required. Prioritize a small set of well-justified tests driven by your equivalence-partition and boundary-value analysis.

1. The Practical Problem

You are testing a small component of a university **Course Registration System** that (i) checks whether a student can register for a course and (ii) calculates the resulting registration fee. **The starter code provides a complete, correct implementation of both functions below in `src/registration.py` — you will not modify this business logic.** Your work is to design and implement a thorough, well-organized `pytest` suite against it, practicing the sequence: requirement → test conditions → test cases → test implementation → execution → analysis.

1.1 Course Registration Rules

A student may register for a course when all of the following hold: current credit load is between 0 and 15 (inclusive); the requested course is between 1 and 4 credits (inclusive); the resulting credit load does not exceed 18; and the prerequisite is met.

```
can_register(current_credits, course_credits, prerequisite_met) # -> bool
```

1.2 Tuition/Registration Fee Rules

When registration is allowed, the fee is: \$100/credit for the first 12 credits, \$75/credit for credits above 12 up to 18. `total_credits` must be between 0 and 18 inclusive; invalid values raise `ValueError`.

```
calculate_registration_fee(total_credits) # -> float, or raises ValueError
```

2. GitHub Project and Submission Requirements [0.5 points]

Create a repository, add a branch named exactly `assignment-1`, and give the instructor (GitHub ID `mdkamrulhasan`) access. All work must be committed/pushed to `assignment-1`; the **latest push timestamp on that branch is your submission date** — do not push to it again after your final submission (use a different branch if you keep working).

3. Set Up the Starter Project [0.5 points]

```
assignment-1/  
|-- src/registration.py (provided, fully implemented)  
|-- tests/test_registration.py  
|-- tests/conftest.py  
|-- pyproject.toml  
|-- README.md
```

Clone the provided starter project, confirm that `pytest` runs cleanly from the project root, and note in `README.md` any environment setup steps needed to run it. `pyproject.toml` is preconfigured so `registration` can be imported directly in your tests (no `src.` prefix) and already registers the markers used in Section 8.

4. Equivalence Partitioning: Decide What to Test [2 points]

For `can_register()`, identify equivalence classes for `current_credits`, `course_credits`, `prerequisite_met`, and the resulting credit-load condition. For `calculate_registration_fee()`, identify valid and invalid classes for `total_credits`.

- (a) **[1.25]** Create an equivalence-partition table identifying each valid and invalid class, based on the *specified behavior*, not arbitrary numerical ranges.
- (b) **[0.75]** Select representative values for the classes and briefly explain why they give useful coverage without testing every input.

5. Boundary-Value Analysis: Focus on the Risky Inputs [2 points]

Use your equivalence classes to identify important boundaries, including at minimum: 0, 15, and 18 credits; 1 and 4 course credits; the transition at 12 credits for the fee; and invalid values just below/above valid ranges.

- (a) **[1]** Identify each boundary and its neighboring values.
- (b) **[0.75]** State the expected behavior for each neighboring value.
- (c) **[0.25]** Briefly explain how a wrong `<`, `<=`, `>`, or `>=` could produce a defect that boundary testing would reveal.

6. Positive and Negative Testing with `pytest` [2.5 points]

- (a) **[0.75]** Positive tests for representative valid registration scenarios (prerequisite met, valid resulting credit load).
- (b) **[0.75]** Negative tests for invalid registration scenarios (prerequisite failure, credit-load violations).
- (c) **[0.5]** Positive tests for the fee calculation, including cases below, at, and above the 12-credit transition.
- (d) **[0.5]** Negative tests for invalid fee inputs (below 0, above 18) using `pytest.raises(ValueError)`.

Name tests by behavior, e.g. `test_registration_rejected_when_total_credits_exceed_18()`.

7. Parametrization and Fixtures [1.5 points]

- (a) **[0.5]** Use `@pytest.mark.parametrize` for groups of similar input/expected-output cases (especially your boundary and equivalence-class tests).
- (b) **[0.5]** Create at least one fixture in `tests/conftest.py` providing reusable test data/setup, used by at least two tests.
- (c) **[0.5]** Use a `yield`-based fixture to demonstrate setup and cleanup of a temporary resource for one test; cleanup must run even if the test fails.

Keep tests independent — no test should depend on another’s execution order or leftover state.

8. Organize and Demonstrate the Test Suite [1 point]

- (a) **[0.4]** `pyproject.toml` already registers four markers — `unit`, `positive`, `negative`, `boundary`. Apply at least two of them to your tests as appropriate (you may register further markers of your own if useful).
- (b) **[0.3]** In `README.md`, document commands for running the full suite and at least two selective subsets using your markers.
- (c) **[0.3]** Demonstrate the full suite passes from the project root; briefly describe how your organization avoids test dependencies/state leakage.

9. Deliverables Checklist [*No additional points*]

Before your final push, confirm the repository has: the provided `src/registration.py` unmodified; `tests/test_registration.py` and `tests/conftest.py`; `pyproject.toml` and `README.md`; your equivalence-partition and boundary-value analyses; positive, negative, and parametrized tests; a fixture with setup/cleanup; registered/used markers; and evidence the full suite passes. Recall: the last push to `assignment-1` is your submission — do not push to it again afterward.

Guiding principle: Equivalence partitioning and boundary-value analysis tell you *what* to test; positive/negative testing describes the *behaviors* to exercise; `pytest` is how you implement, organize, and run it all.